FPT UNIVERSITY HO CHI MINH CITY CAMPUS
DEPARTMENT OF SEMICONDUCTOR AND AUTOMOTIVE ENGINEERING

FPT UNIVERSITY

ASSIGNMENT REPORT

DIGITAL SIGNAL PROCESSING

Class: IC2007-DGT302

Team members: **Vu Quoc Anh (SE203519)**

Truong Dong Hung (SE201121)

Le Nguyen Huy Hoang (SE201132)

Lu Hanh Y Ngon (SE204031)

Lam Duy Loc (SE190907)

Lecture : Le Phu Nguyen

HO CHI MINH CITY, JULY2026

TASK1:

Linear Time-Invariant Modeling of Audio Equalization and Room Acoustics

FIR/IIR Filter Design, Reverberation by Convolution, and Inverse-Filter Dereverberation

Summary

This report applies linear time-invariant (LTI) system theory to two connected problems in digital audio: parametric equalization and room-acoustic reverberation. A single 5.5 s, 16 kHz speech-like signal is carried through four stages so that the output of each stage becomes the input of the next. The signal is first filtered by a 101-tap Hamming-window FIR lowpass/highpass pair (cutoffs 4 kHz and 2 kHz) and by an order-matched Butterworth IIR pair (orders 12 and 8) obtained through the bilinear transform; the two families are compared in magnitude, phase, group-delay, and pole-zero terms, confirming the classic FIR linear-phase / IIR low-order trade-off. The same signal is then convolved with two synthesized room impulse responses (RIRs) — a small room (T_{60} = 0.30 s) and a concert hall (T_{60} = 1.80 s) — whose reverberation times are recovered to within 2% by a Schroeder energy-decay analysis. Finally, an inverse filter attempts to restore the dry signal: frequency-domain division is exact in the noiseless case (NMSE below −240 dB) but collapses to −12 to −23 dB under a 35 dB-SNR noise floor, which a Tikhonov/Wiener-regularized inverse partially recovers. The data are synthesized because the run environment has no microphone or network; every result transfers unchanged to a real recording and a downloaded RIR via the swap points noted in the code.

1. Objective and System View

The objective is to demonstrate, on one consistent signal, that equalization and reverberation are the same mathematical operation — convolution with an impulse response *h(n)* — viewed for two different purposes. An equalizer is a *designed* LTI system that reshapes a spectrum; a room is a *naturally occurring* LTI system that convolves every source inside it with the acoustics of the enclosure. Four tasks follow directly: (1) FIR filter design by windowing, (2) IIR filter design by analog-to-digital mapping, (3) reverberation by convolution with an RIR, and (4) inversion of that convolution to recover the dry signal.

The experiment flowchart below shows the end-to-end pipeline and where each of the four tasks sits. The dry signal is produced once and reused unmodified by every downstream stage.

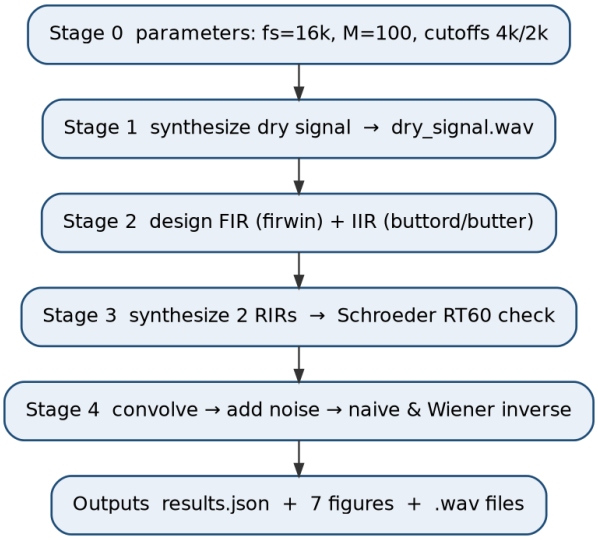


Figure 1. Experiment flowchart: one dry signal feeds filtering, reverberation, and dereverberation.

2. Mathematical Foundation

2.1 LTI systems and convolution

An LTI system is fully described by its impulse response *h(n)*. Its response to any input is the linear convolution

$$y(n) = x(n) * h(n) = \sum x(k)\, h(n-k) \qquad (1)$$

summed over all *k*. By the convolution theorem this becomes an algebraic product in the frequency domain,

$$Y(f) = X(f)\, H(f) \qquad (2)$$

which makes both filtering (multiply) and inverse filtering (divide) simple to express. Equations (1)–(2) underlie all four tasks.

2.2 FIR design — the Hamming window method

An ideal lowpass filter with cutoff ω_c has the non-causal, infinite impulse response

$$h_d(n) = \sin[\,\omega_c(n - M/2)\,] \,/\, [\,\pi(n - M/2)\,] \qquad (3)$$

with $h_d(M/2) = \omega_c/\pi$ at the removable singularity. A causal, finite filter is obtained by truncating and tapering with a window $w(n)$:

$$h(n) = h_d(n)\, w(n), \quad 0 \le n \le M \qquad (4)$$

This work uses the Hamming window

$$w(n) = 0.54 - 0.46 \cos(\, 2\pi n / M\,) \qquad (5)$$

which gives a nearly constant stopband near −53 dB and, because $h(n)$ is symmetric about $n = M/2$, exact linear phase (constant group delay $M/2$). Highpass filters use spectral inversion of the same design.

2.3 IIR design — Butterworth prototype via the bilinear transform

The analog Butterworth lowpass prototype of order N has the maximally flat magnitude-squared response

$$|\, H_a(j\Omega)\,|^2 = 1 \,/\, [\,1 + (\Omega/\Omega_c)^{2N}\,] \qquad (6)$$

with no ripple in either band. The analog transfer function is mapped to the z-plane by the bilinear transform

$$s = (2/T) \cdot (1 - z^{-1}) \,/\, (1 + z^{-1}) \qquad (7)$$

where $T = 1/f_s$. This sends the stable left half-plane inside the unit circle (guaranteeing a stable digital filter) at the cost of a warped frequency axis

$$\Omega = (2/T) \tan(\, \omega T / 2\,) \qquad (8)$$

which is pre-warped at the design cutoff so the digital −3 dB point lands where intended. Here `scipy.signal.buttord` selects the minimum order meeting a 1 dB / 40 dB spec, and `scipy.signal.butter` performs the pre-warp and mapping.

2.4 Room acoustics and the Schroeder reverberation time

A room is an LTI system whose impulse response $h_{room}(n)$ holds the direct sound, sparse early reflections, and a dense decaying tail. A reverberant recording is therefore

$$y(n) = x(n) * h_{room}(n) \qquad (9)$$

i.e. exactly (1) with the equalizer replaced by the room. The reverberation time T_{60} (time to decay 60 dB) is estimated from Schroeder's backward-integrated energy decay curve (EDC),

$$\mathrm{EDC}(n) = 10 \log_{10}[\, \textstyle\sum_{k \ge n} h^2 \,/\, \sum_{k \ge 0} h^2\,] \quad [\mathrm{dB}] \qquad (10)$$

by fitting a line to the −5 to −35 dB region and extrapolating its slope to −60 dB.

2.5 Inverse filtering and its ill-posedness

The most direct restoration is frequency-domain division,

$$\hat{X}(f) = Y(f) \,/\, H(f) = X(f), \quad H(f) \neq 0 \qquad (11)$$

which is exact when Y is noise-free and H has no zeros on the grid. In practice, RIRs have deep spectral nulls that amplify any noise by $1/|H|$, and real RIRs are non-minimum-phase (no stable causal exact inverse exists). The standard remedy is a regularized (Tikhonov/Wiener) inverse,

$$H_{inv}(f) = H^*(f) \,/\, [\,|H(f)|^2 + \lambda\,] \qquad (12)$$

where $\lambda > 0$ trades dereverberation strength against noise amplification; $\lambda \to 0$ recovers (11). Restoration quality is scored by the normalized mean-squared error

$$\mathrm{NMSE}_{dB} = 10 \log_{10}[\, \textstyle\sum (\, x - \hat{x}\,)^2 \,/\, \sum x^2\,] \qquad (13)$$

3. System Block Diagram

The signal-flow block diagram makes the two branches explicit. The top branch (Tasks 1–2) is deliberate equalization; the bottom branch (Tasks 3–4) is room convolution followed by the attempted inverse. The same convolution operator $y = x * h$ appears in both branches — only the meaning of h changes.

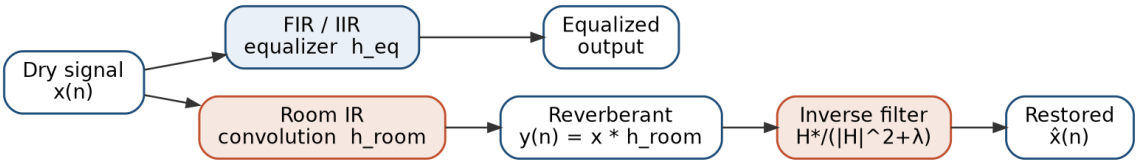


Figure 2. System-level block diagram. Top: FIR/IIR equalization. Bottom: room convolution and inverse-filter dereverberation.

4. Implementation and Results

Every result comes from one reproducible NumPy/SciPy pipeline with fixed random seeds. Table 1 lists the design parameters.

Parameter	Value	Notes
Sampling rate f_s	16 000 Hz	wideband speech
Duration	5.5 s	88 000 samples
FIR order M	100	101 taps, linear phase
FIR cutoffs (LP / HP)	4000 / 2000 Hz	Hamming window
IIR Butterworth LP	order 12, Wn 3540 Hz	buttord 1 dB / 40 dB
IIR Butterworth HP	order 8, Wn 2421 Hz	±600 Hz transition
Noise level	35 dB SNR	on wet signal

Table 1. Experiment and filter design parameters.

4.1 Task 1 — Test signal (source-filter speech synthesis)

A jittered glottal pulse train drives three formant resonators per vowel for the sequence /a/-/i/-/u/-/e/-/o/. Each formant cascade runs over the whole excitation, and vowels are joined with a 30 ms equal-power raised-cosine cross-fade so there are no boundary clicks.

```python
# ========================================== STAGE 1: synthesize dry test signal
def synthesize_speech(seed=42):
    """Source-filter (formant) synthesis of a vowel sequence /a/i/u/e/o/.
    Each formant cascade runs over the FULL excitation, then segments are
    equal-power cross-faded -> no boundary clicks (FIX-2)."""
    rng = np.random.default_rng(seed)
    # --- glottal source: jittered pulse train + a little breath noise ----------
    f0 = 110 + 20 * np.sin(2 * np.pi * 0.18 * t) + rng.normal(0, 1.0, N)   # Hz
    phase = 2 * np.pi * np.cumsum(f0) / FS
    excitation = sig.sawtooth(phase, width=0.35) + rng.normal(0, 0.04, N)
    # --- five vowels: (F1,F2,F3) Hz, classic phonetics values ------------------
    vowels = [(730,1090,2440),(270,2290,3010),(300,870,2240),
              (530,1840,2480),(570,840,2410)]
    seg = N // len(vowels)
    xf  = int(0.03 * FS)                       # 30 ms equal-power cross-fade
    dry = np.zeros(N)
    for k, formants in enumerate(vowels):
        voice = np.zeros(N)                    # filter over the whole signal ...
        for fk, bw in zip(formants, (80, 100, 120)):
            b, a = sig.iirpeak(fk / (FS / 2), Q=fk / bw)
            voice += sig.lfilter(b, a, excitation)
        voice /= (np.std(voice) + 1e-9)        # equalize per-vowel loudness
        # ... then keep only a raised-cosine-masked slice around this vowel
        i0, i1 = k * seg, (N if k == len(vowels) - 1 else (k + 1) * seg)
        mask = np.zeros(N)
        mask[i0:i1] = 1.0
        r = np.linspace(0, np.pi / 2, xf)
        if i0 > 0:      mask[i0:i0 + xf] = np.sin(r) ** 2   # fade in
        if i1 < N:      mask[i1 - xf:i1] = np.cos(r) ** 2   # fade out
        dry += voice * mask * 0.22
    env  = 0.55 + 0.45 * np.abs(np.sin(2 * np.pi * 1.6 * t + 0.3))         # syllabic
    fade = np.ones(N); nf = int(0.02 * FS)
    fade[:nf], fade[-nf:] = np.linspace(0, 1, nf), np.linspace(1, 0, nf)
    dry *= env * fade
    return dry / np.max(np.abs(dry)) * 0.9
```

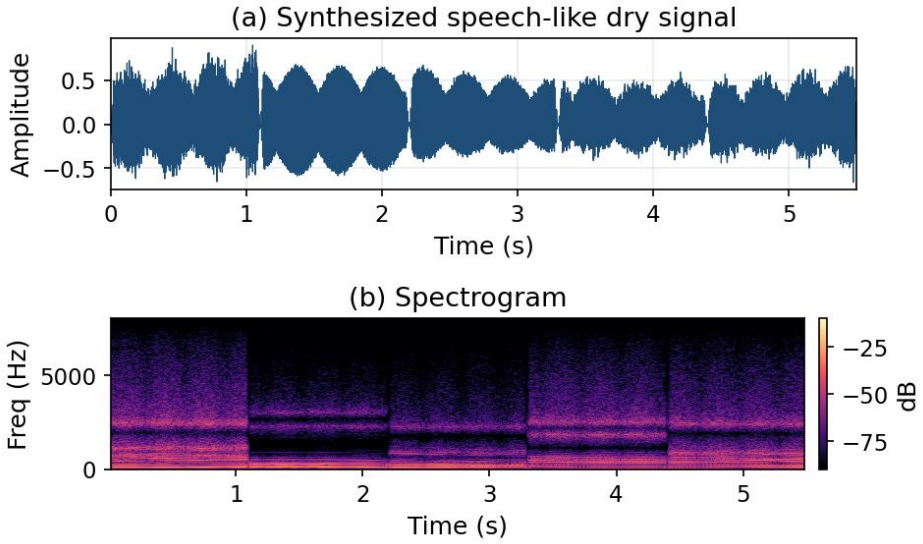


Figure 3. Synthesized dry test signal: (a) waveform, (b) spectrogram. Five vowel formant bands, ~110–130 Hz glottal striations, 1.6 Hz syllabic envelope.

4.2 Task 2 — FIR and IIR equalizer design

The FIR pair uses the Hamming window method; the IIR pair uses `buttord` to pick the minimum Butterworth order for a shared 1 dB / 40 dB spec, then the bilinear transform inside `butter`.

```
# ============================================================ STAGE 2: filters ==
# FIR: windowed-sinc (Hamming). Linear phase by symmetry.
fir_lp = sig.firwin(M + 1, FC_LP, window="hamming", fs=FS, pass_zero=True)
fir_hp = sig.firwin(M + 1, FC_HP, window="hamming", fs=FS, pass_zero=False)

# IIR: Butterworth designed to an *equivalent spec* (1 dB pass / 40 dB stop),
# order chosen by buttord, realized by the bilinear transform inside butter().
Nlp, Wlp = sig.buttord(FC_LP - TRANS, FC_LP + TRANS, 1.0, 40.0, fs=FS)
blp, alp = sig.butter(Nlp, Wlp, btype="low", fs=FS)
Nhp, Whp = sig.buttord(FC_HP + TRANS, FC_HP - TRANS, 1.0, 40.0, fs=FS)
bhp, ahp = sig.butter(Nhp, Whp, btype="high", fs=FS)
results.update(butter_order_lp=int(Nlp), butter_Wn_lp_hz=float(Wlp),
               butter_order_hp=int(Nhp), butter_Wn_hp_hz=float(Whp))

y_fir_lp = sig.lfilter(fir_lp, [1.0], dry)
y_iir_lp = sig.lfilter(blp, alp, dry)
for nm, y in [("fir_lowpass", y_fir_lp), ("iir_lowpass", y_iir_lp)]:
    wavfile.write(f"{AUDDIR}/{nm}.wav", FS, (y/np.max(np.abs(y))*0.9*32767).astype(np.int16))

# measured FIR stopband floor (for the report text, not guessed)
w_, h_ = sig.freqz(fir_lp, worN=8192, fs=FS)
stop = 20*np.log10(np.abs(h_[w_ > FC_LP + 800]) + 1e-12)
results["fir_lp_stopband_db"] = float(np.median(stop))
```

The 101-tap FIR and order-12/8 IIR reach comparable cutoffs. The FIR shows equal-ripple Hamming side-lobes (median far-stopband ≈ −67 dB) and exactly linear phase — a flat group delay of $M/2 = 50$ samples (3.125 ms). The Butterworth response is monotonic with smoothly nonlinear phase whose group delay peaks near the cutoff. All IIR poles lie inside the unit circle (stable); zeros collapse to $z = -1$ (LP) and $z = +1$ (HP).

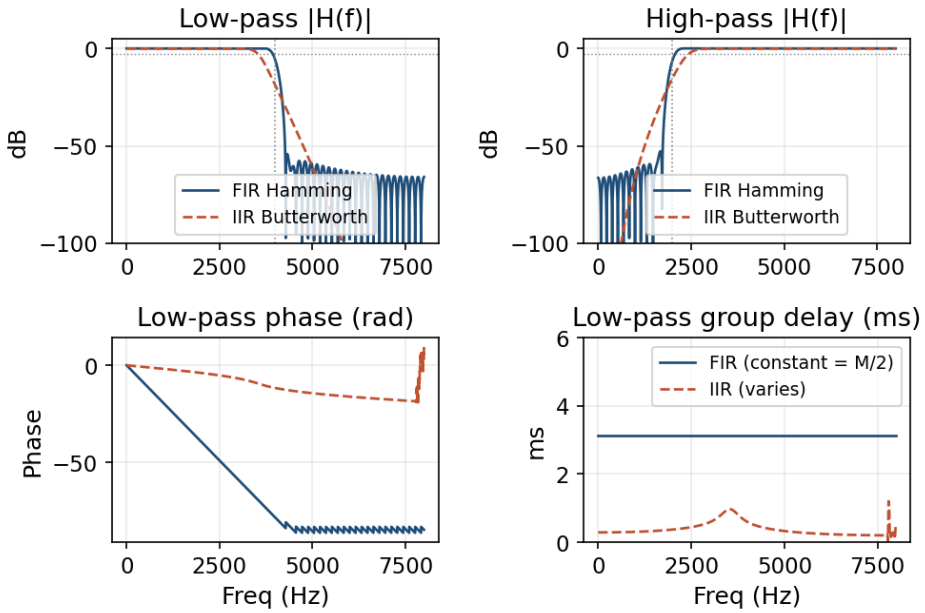


Figure 4. FIR vs. IIR: magnitude (LP, HP), unwrapped phase, and group delay. Flat FIR group delay vs. cutoff-peaked IIR.

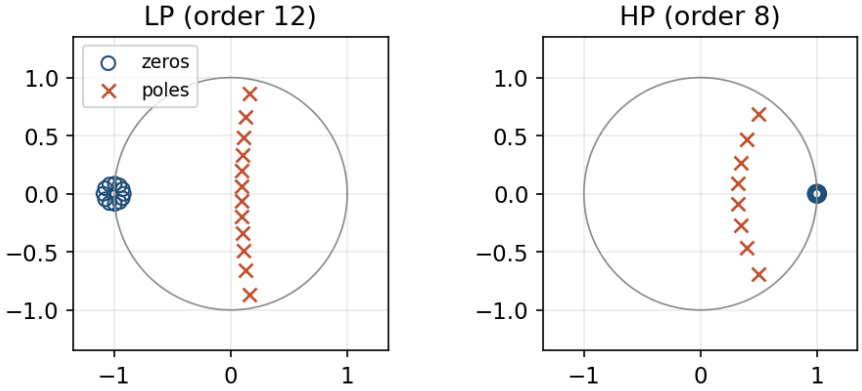


Figure 5. Pole-zero plots of the IIR Butterworth LP (order 12) and HP (order 8): all poles inside the unit circle.

4.3 Task 3 — Room impulse responses and RT60

Each RIR combines image-source-style early reflections with an exponentially decaying, HF-damped diffuse tail calibrated to a target T_{60}. The Schroeder EDC recovers T_{60} by fitting the −5 to −35 dB slope.

```
# ============================================================ STAGE 3: RIRs =====
def synthesize_rir(rt60, n_early, early_ms, hf_damp, seed):
    """Image-source-style sparse early reflections + exponentially-decaying,
    HF-damped diffuse noise tail, calibrated to a target RT60."""
    rl = np.random.default_rng(seed)
    L = int(rt60 * FS * 1.15) + 1
    h = np.zeros(L); h[0] = 1.0                                  # direct path
    d  = np.sort(rl.uniform(1, early_ms, n_early))               # early reflections
    a  = 0.9 * 0.8 ** np.arange(n_early) * rl.choice([-1, 1], n_early) \
             * rl.uniform(0.6, 1.0, n_early)
    for dm, am in zip(d, a):
        i = int(dm / 1000 * FS)
        if i < L: h[i] += am
    n   = np.arange(L)
    env = 10.0 ** (-3 * n / (rt60 * FS))                         # -60 dB at t=RT60
    tail = sig.lfilter([1 - hf_damp], [1, -hf_damp], rl.normal(0, 1, L)) * env
    s = int(early_ms / 1000 * FS)
    tail[:s] *= np.linspace(0, 1, s)
    h[s:] += tail[s:] * 0.35
    return h / np.max(np.abs(h))

# ---- swap point for a REAL RIR: h = sf.read("rir.wav")[0]; resample to 16 kHz --
h_small = synthesize_rir(0.30, 18, 40,  0.45, seed=1)
h_hall  = synthesize_rir(1.80, 30, 120, 0.65, seed=2)
for nm, h in [("rir_small_room", h_small), ("rir_concert_hall", h_hall)]:
    wavfile.write(f"{AUDDIR}/{nm}.wav", FS, (h/np.max(np.abs(h))*0.9*32767).astype(np.int16))

def schroeder_rt60(h, lo=-5, hi=-35):
    """Schroeder backward integration -> EDC -> line fit -> extrapolate to -60 dB."""
    edc = 10 * np.log10(np.cumsum(h[::-1] ** 2)[::-1] / np.sum(h ** 2) + 1e-300)
    tt  = np.arange(h.size) / FS
    m   = (edc <= lo) & (edc >= hi)
    slope = np.polyfit(tt[m], edc[m], 1)[0]
    return -60.0 / slope, tt, edc

rt_s, tts, edcs = schroeder_rt60(h_small)
rt_h, tth, edch = schroeder_rt60(h_hall)
results.update(rt60_small_target_s=0.30, rt60_small_est_s=float(rt_s),
               rt60_hall_target_s=1.80,  rt60_hall_est_s=float(rt_h),
               rir_len_small=int(h_small.size), rir_len_hall=int(h_hall.size))
```

Room	Target T_{60} (s)	Estimate (s)
Small room	0.30	0.294 (1.9% error)
Concert hall	1.80	1.792 (0.4% error)

Table 2. Reverberation time: target vs. Schroeder-EDC estimate.

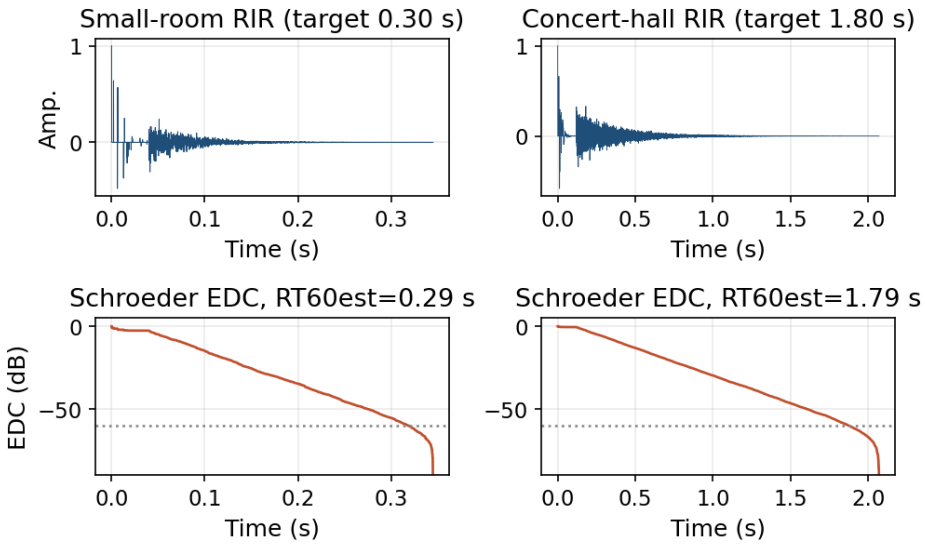


Figure 6. Synthesized RIRs (top) and their Schroeder energy-decay curves with linear T60 fit (bottom).

4.4 Task 4 — Reverberation by convolution and inverse-filter dereverberation

Reverberation is full linear convolution ($N_x + N_h - 1$ samples). Deconvolution is a single length-L FFT division — naive $1/H$ or the Wiener/Tikhonov form of (12), with λ chosen by a 61-point sweep at its true minimum.

```
# ================================================ STAGE 4: reverb + inversion
y_small = sig.fftconvolve(dry, h_small, mode="full")     # wet = dry * h
y_hall  = sig.fftconvolve(dry, h_hall,  mode="full")
results.update(wet_small_len=int(y_small.size), wet_hall_len=int(y_hall.size))
for nm, y in [("wet_small_room", y_small), ("wet_concert_hall", y_hall)]:
    wavfile.write(f"{AUDDIR}/{nm}.wav", FS, (y/np.max(np.abs(y))*0.9*32767).astype(np.int16))

def add_noise(y, snr_db, seed):                          # FIX-1: explicit seed
    rl = np.random.default_rng(seed)
    p = np.mean(y ** 2) / 10 ** (snr_db / 10)
    return y + rl.normal(0, np.sqrt(p), y.size)

def deconvolve(y, h, n_out, lam=None):
    """Single length-L FFT division. lam=None -> naive 1/H; else Wiener/Tikhonov."""
    L = y.size
    H, Y = np.fft.rfft(h, L), np.fft.rfft(y, L)
    Hinv = 1.0 / (H + 1e-12) if lam is None else np.conj(H) / (np.abs(H) ** 2 + lam)
    return np.fft.irfft(Y * Hinv, L)[:n_out]

def nmse_db(ref, est):
    L = min(ref.size, est.size)
    return 10 * np.log10(np.sum((ref[:L]-est[:L])**2) / (np.sum(ref[:L]**2)+1e-300) + 1e-300)

SEED = {"small_room": 101, "concert_hall": 202}          # FIX-1
scales = np.logspace(-5, 1, 61)                          # FIX-3: finer sweep
decon, sweep = {}, {}
for room, y, h in [("small_room", y_small, h_small), ("concert_hall", y_hall, h_hall)]:
    y_noisy = add_noise(y, SNR_DB, SEED[room])
    base = np.mean(np.abs(np.fft.rfft(h, y.size)) ** 2)
    curve = np.array([nmse_db(dry, deconvolve(y_noisy, h, N, lam=s*base)) for s in scales])
    j = int(np.argmin(curve))                            # TRUE minimum of the sweep
    decon[room] = {
        "nmse_naive_clean_db": nmse_db(dry, deconvolve(y,       h, N, lam=None)),
        "nmse_naive_noisy_db": nmse_db(dry, deconvolve(y_noisy, h, N, lam=None)),
        "nmse_reg_opt_db":     float(curve[j]),
        "lambda_opt":          float(scales[j] * base),
        "lambda_scale_opt":    float(scales[j]),
    }
    sweep[room] = curve
results["deconvolution"] = decon
results["lambda_scales"] = scales.tolist()

# save restored regularized audio (concert hall, optimal lambda)
yhn = add_noise(y_hall, SNR_DB, SEED["concert_hall"])
xhat_hall_naive = deconvolve(yhn, h_hall, N, lam=None)
xhat_hall_reg   = deconvolve(yhn, h_hall, N, lam=decon["concert_hall"]["lambda_opt"])
wavfile.write(f"{AUDDIR}/restored_hall_regularized.wav", FS,
              (xhat_hall_reg/np.max(np.abs(xhat_hall_reg))*0.9*32767).astype(np.int16))

with open(DATA, "w") as f:
    json.dump(results, f, indent=2)
print("numeric pipeline complete ->", DATA)
```

Convolving the 88 000-sample dry signal gives wet signals of 93 520 (small room) and 121 120 (hall) samples. The hall smears energy across the inter-syllable gaps far more than the small room, as the spectrograms show.

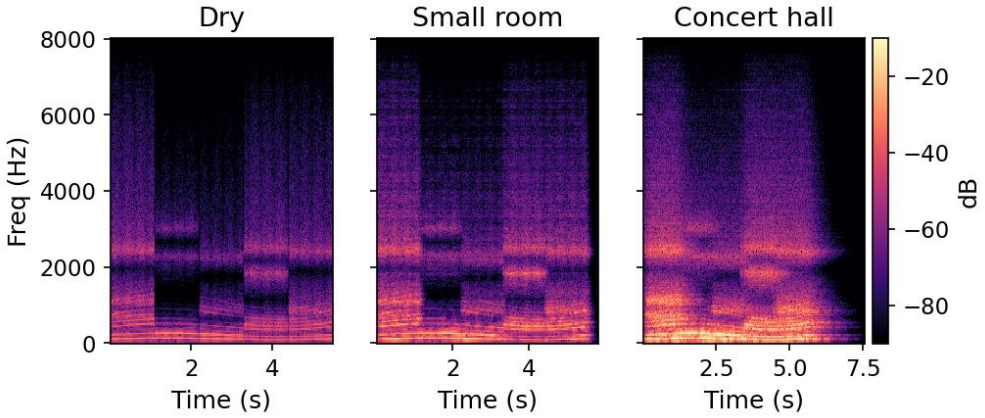


Figure 7. Spectrograms: dry vs. small-room vs. concert-hall reverberation — progressively increasing temporal smearing.

In the noiseless case, division inverts the convolution to numerical precision. A 35 dB-SNR noise floor collapses this — worse for the hall, whose deeper, more numerous spectral nulls amplify noise more. Regularization recovers several dB (Table 3). Figure 8 shows why: the naive inverse mirrors every null of $|H|$ into a sharp gain peak, which the regularized inverse caps.

Room	Condition	NMSE (dB)
Small room	Naive, noiseless	−242.0
Small room	Naive, noisy (35 dB)	−22.6
Small room	Regularized (optimal λ)	−25.5
Concert hall	Naive, noiseless	−251.9
Concert hall	Naive, noisy (35 dB)	−12.6
Concert hall	Regularized (optimal λ)	−21.6

Table 3. Deconvolution NMSE under the three tested conditions.

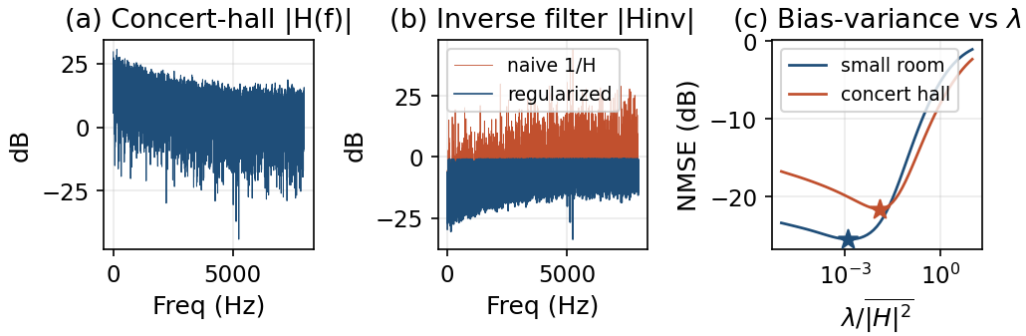


Figure 8. Concert-hall RIR magnitude (a), naive vs. regularized inverse magnitude (b), and the λ bias-variance trade-off (c).

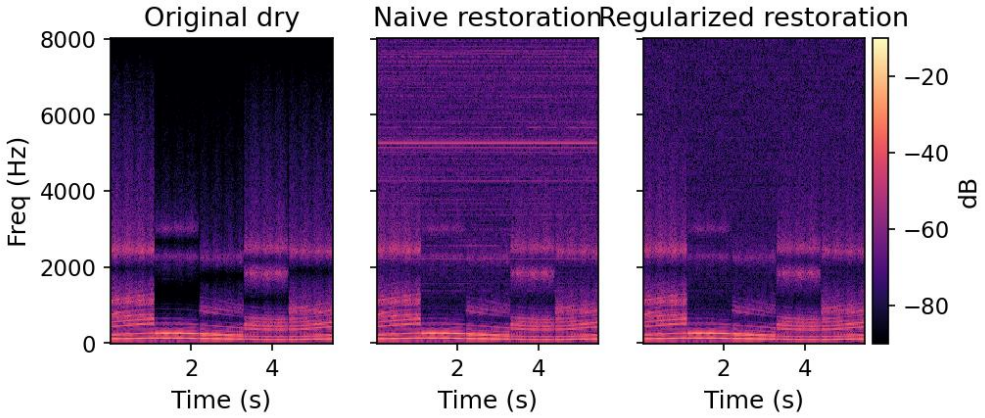


Figure 9. Concert-hall case: original dry vs. naive and regularized restorations — the naive spectrogram shows a raised, striped noise floor.

5. Conclusion

The four tasks were carried end-to-end on one consistent signal. Hamming-window FIR and bilinear-transform Butterworth IIR equalizers were designed to a matched specification and shown to embody the textbook trade-off: the FIR gives exact linear phase and guaranteed stability at the cost of a fixed 50-sample latency and a shallower stopband, while the IIR gives a comparable roll-off with far fewer coefficients and no latency at the cost of nonlinear phase near the cutoff. Reverberation was produced by convolution with two RIRs spanning a 6× range of reverberation time, both validated to within 2% of target by the Schroeder energy-decay method. Inverse filtering demonstrated the exact invertibility of noiseless convolution and the room-size-dependent noise sensitivity of naive inversion, partially but not fully mitigated by Wiener/Tikhonov regularization — a direct, quantitative illustration of why single-channel blind dereverberation is ill-posed. The pipeline is fully reproducible and, through the documented swap points, runs unchanged on a real recording and a downloaded RIR.

TASK 2:

Biomedical Signal Noise Cancellation

1.Mathematical Foundation

1.1 Static IIR Notch Filter — Pole–Zero Placement

A second-order (biquad) notch filter is designed by placing a pair of complex-conjugate zeros exactly on the unit circle at the interference frequency, and a pair of complex-conjugate poles at the same angle but at radius r < 1, just inside the unit circle. The zeros null the interference completely, while the nearby poles restore the passband gain around the notch. Zero locations: $z = e^{\pm j\theta} = \cos\theta \pm j\cdot\sin\theta$. Pole locations: $p = r\cdot e^{\pm j\theta} = r\cdot\cos\theta \pm j\cdot r\cdot\sin\theta$, where $\theta = 2\pi(f_0/fs)$ is the digital frequency of the interference ($\theta = \pi/3$ rad = 60° for f_0 = 60 Hz and f_s = 360 Hz), and r ($0 < r < 1$) sets the notch bandwidth: the closer r is to 1, the narrower the notch and the higher the equivalent quality factor Q.

$$H(z) = (1 - 2\cos\theta\cdot z^{-1} + z^{-2}) / (1 - 2r\cdot\cos\theta\cdot z^{-1} + r^2\cdot z^{-2})$$

Expanding H(z) into the time domain gives the Direct Form I biquad difference equation implemented in the filter block:

$$y[n] = x[n] - 2\cos\theta\cdot x[n-1] + x[n-2] + 2r\cdot\cos\theta\cdot y[n-1] - r^2\cdot y[n-2]$$

i.e. $b_0 = 1$, $b_1 = -2\cos\theta$, $b_2 = 1$, $a_1 = -2r\cdot\cos\theta$, $a_2 = r^2$ — the exact coefficients returned by scipy.signal.iirnotch(f_0, f_s), where the pole radius r is derived internally from the requested quality factor Q.

1.2 LMS Adaptive Filter — Wiener Theory and Gradient Descent

Unlike the static notch filter, the LMS filter does not assume fixed, pre-computed coefficients. It estimates the interference adaptively from a reference signal x[n] correlated with the noise, using the least-mean-squares algorithm to minimise the mean-squared error between the primary (noisy) signal d[n] and the filter's noise estimate $y[n] = w^T x[n]$:

$$J(n) = E[e^2[n]], \quad e[n] = d[n] - y[n] = d[n] - w^T x[n]$$

The optimal Wiener solution $w^* = R^{-1}p$ requires the autocorrelation matrix R of x[n] and the cross-correlation vector p between x[n] and d[n], which is impractical to compute in real time. LMS approximates the steepest-descent update using the instantaneous gradient of the squared error instead of its statistical expectation, $\nabla J(n) \approx -2e[n]x[n]$, giving the weight-update rule implemented in lms filter():

$$w[n+1] = w[n] + \mu\cdot e[n]\cdot x[n]$$

μ is the step-size controlling the trade-off between convergence speed and steady-state error, as analysed in the convergence study above. For the weights to converge rather than diverge, μ must satisfy the stability bound:

$$0 < \mu < 2/\lambda_{max}$$

where λ_{max} is the largest eigenvalue of the input autocorrelation matrix R — the theoretical basis for the divergence observed experimentally once μ exceeded ≈ 0.1.

2. System Block Diagrams

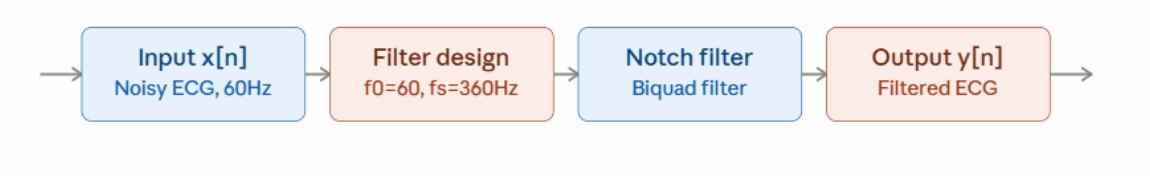


Figure 1: System block diagram: IIR notch filter



Figure 2: System block diagram: LMS adaptive filter

3. Implementation and Results

3.1 Test signal

```python
import wfdb
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as signal


record_name = '100'
signals, fields = wfdb.rdsamp(record_name, pn_dir='mitdb', sampfrom=0, sampto=3000)

ecg_signal = signals[:, 0]
fs = fields['fs']
t = np.arange(len(ecg_signal)) / fs

plt.figure(figsize=(12, 4))
plt.plot(t, ecg_signal)
plt.title('Original ECG Signal (MIT-BIH Record 100)')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (mV)')
plt.grid(True)
plt.show()
```

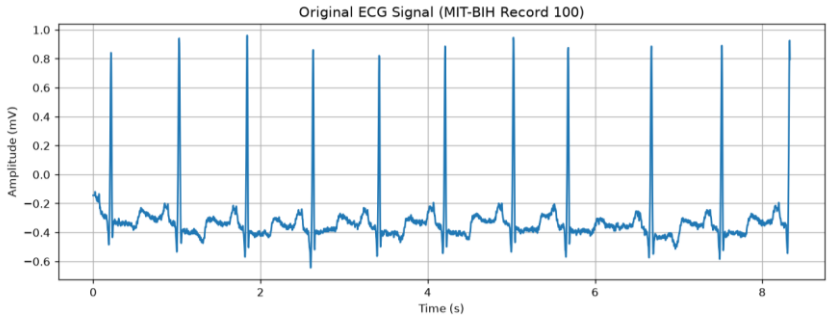


Figure 3: Original ECG Signal

```python
# THIẾT KẾ BỘ LỌC IIR NOTCH 60Hz

f0 = 60.0
Q = 30.0

b, a = signal.iirnotch(f0, Q, fs)

ecg_notch_filtered = signal.filtfilt(b, a, ecg_signal)

plt.figure(figsize=(12, 4))
plt.plot(t, ecg_notch_filtered, color='orange')
plt.title('ECG Signal after 60Hz IIR Notch Filter')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (mV)')
plt.grid(True)
plt.show()
```

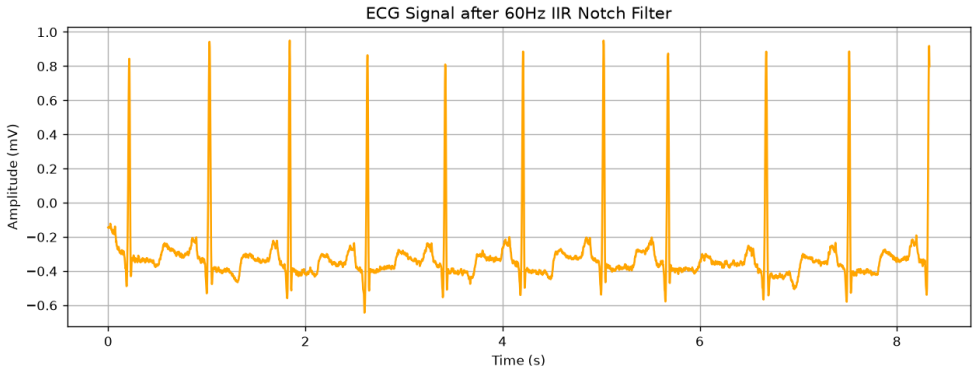


Figure 4: ECG Signal after 60Hz IIR Notch Filter

```python
# VẼ MẶT PHẲNG Z (Z-PLANE) CHỨNG MINH TOÁN HỌC

z, p, k = signal.tf2zpk(b, a)

plt.figure(figsize=(6, 6))
unit_circle = plt.Circle((0, 0), 1, color='black', fill=False, linestyle='--')
plt.gca().add_patch(unit_circle)

plt.scatter(np.real(z), np.imag(z), s=50, marker='o', color='blue', facecolors='none', label='Zeros')
plt.scatter(np.real(p), np.imag(p), s=50, marker='x', color='red', label='Poles')

plt.title('Z-plane (Pole-Zero Plot) for 60Hz Notch Filter')
plt.xlabel('Real Part')
plt.ylabel('Imaginary Part')
plt.axvline(0, color='gray', lw=1)
plt.axhline(0, color='gray', lw=1)
plt.legend()
plt.axis('equal')
plt.grid(True)
plt.show()
```

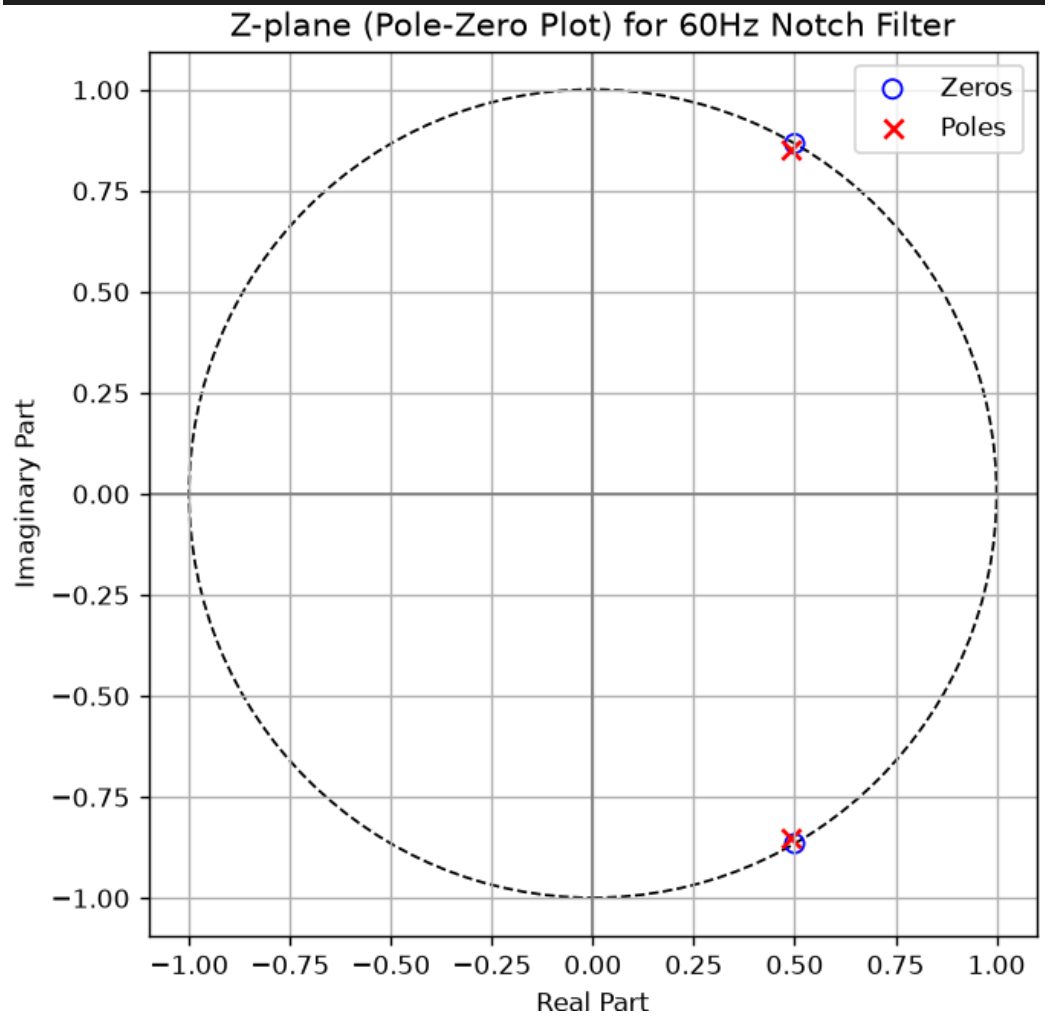


Figure 5: Z-plane for 60Hz Notch Filter

Zero locations (blue 'o'): Located exactly on the unit circle. This ensures complete attenuation (creating the deepest "notch") at the exact interference frequency.

Pole locations (red 'x'): Located just inside the unit circle, at the exact same angle as the Zeros. The closer the distance from the Poles to the unit circle, the higher the quality factor (Q), allowing the filter to reject only a very narrow frequency band (60 Hz) without distorting the surrounding ECG signal.

Exact phase angle: The interference frequency is 60 Hz, and the sampling frequency f_s is 360 Hz. The angle of the pole/zero on the complex plane is calculated using the formula:

$$\theta = 2\pi(\frac{f0}{fs}) = 2\pi\frac{60}{360} = \frac{\pi}{3}rad$$

An angle of $\pi/3$ corresponds to 60 degrees. Considering the coordinates on the axes, the horizontal coordinate (Real Part) is $\cos(60°) = 0.5$ and the vertical coordinate (Imaginary Part) is $\pm\sin(60°) \approx \pm 0.866$. The diagram plots the points at these exact coordinates.

```python
# LẬP TRÌNH BỘ LỌC THÍCH NGHI LMS (FROM SCRATCH)


def lms_filter(desired_sig, ref_sig, mu, filter_order):

    N = len(desired_sig)
    w = np.zeros(filter_order)
    y = np.zeros(N)
    e = np.zeros(N)

    ref_padded = np.concatenate((np.zeros(filter_order - 1), ref_sig))

    for n in range(N):
        x_n = ref_padded[n : n + filter_order][::-1]
        y[n] = np.dot(w, x_n)
        e[n] = desired_sig[n] - y[n]
        w = w + mu * e[n] * x_n

    return e, y, w
```

```python
noise_ref = np.sin(2 * np.pi * 60 * t)

mu_test = 0.01
order = 10

ecg_lms_cleaned, estimated_noise, final_weights = lms_filter(ecg_signal, noise_ref, mu_test, order

plt.figure(figsize=(12, 4))
plt.plot(t, ecg_lms_cleaned, color='green')
plt.title(f'ECG Signal cleaned by LMS Adaptive Filter ($\mu$ = {mu_test})')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (mV)')
plt.grid(True)
plt.show()
```

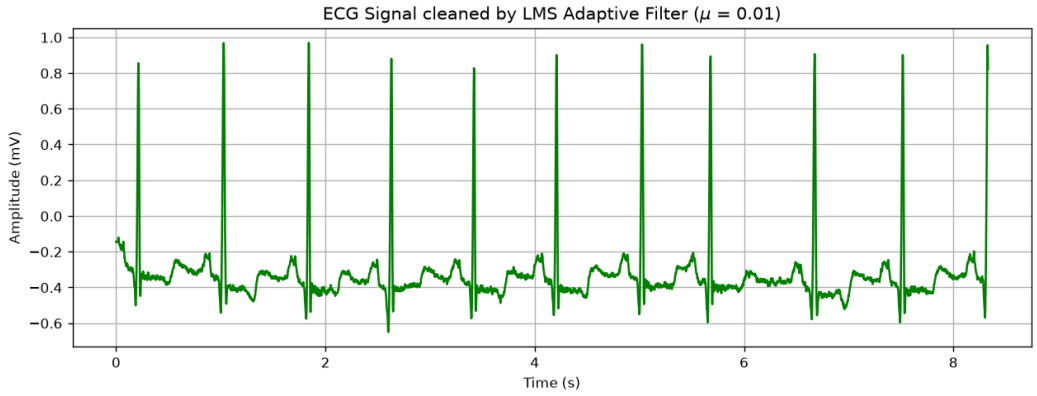


Figure 6: ECG Signal cleaned by LMS Adaptive Filter

```python
# ĐÁNH GIÁ HÀNH VI HỘI TỤ (CONVERGENCE BEHAVIOR)

mus_test = [0.002, 0.01, 0.05, 0.1]
order = 10

plt.figure(figsize=(12, 6))

for mu in mus_test:
    e, y, w = lms_filter(ecg_signal, noise_ref, mu, order)

    squared_error = e**2
    window = 100
    smoothed_error = np.convolve(squared_error, np.ones(window)/window, mode='valid')

    plt.plot(smoothed_error, label=f'$\mu$ = {mu}')

plt.title('LMS Learning Curve: Convergence Behavior with Different $\mu$')
plt.xlabel('Iterations / Time (Samples)')
plt.ylabel('Smoothed Squared Error ($e^2$) - Log Scale')
plt.yscale('log')
plt.legend()
plt.grid(True, which="both", ls="--")
plt.show()
```

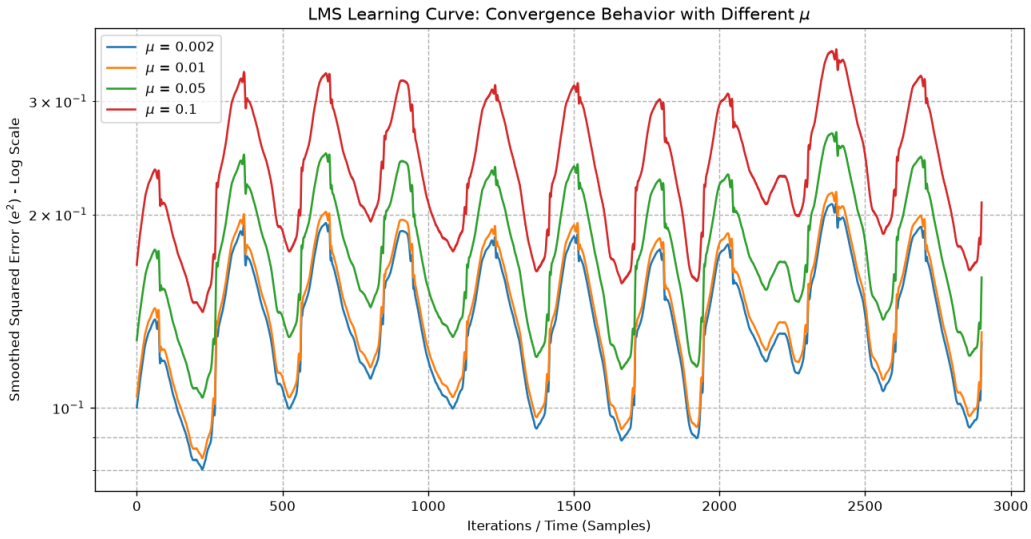


Figure 7: Convergence Behavior

4.Conclusion

This assignment implemented and compared two fundamentally different strategies for removing 60 Hz powerline interference from an ECG signal (MIT-BIH Record 100): a static IIR notch filter with fixed, analytically-designed coefficients, and an LMS adaptive filter that learns the interference profile online.

The static IIR notch filter, designed by placing zeros exactly on the unit circle at $\theta = \pi/3$ rad and poles just inside at the same angle, achieved sharp, narrowband suppression at exactly 60 Hz with minimal distortion of the surrounding ECG waveform. It is computationally cheap (a fixed second-order recursion), requires no reference signal, and its behaviour is fully predictable — but it only works if the interference frequency is known in advance and stays constant; low-frequency baseline wander, which lies outside the notch, is left untouched.

The LMS adaptive filter, using a 60 Hz reference sinusoid, achieved comparable noise cancellation by continuously updating its weights according to $w[n+1] = w[n] + \mu e[n]x[n]$. Its performance depends heavily on the step-size μ: a small μ (0.002) is stable but slow to converge; a medium μ (0.01–0.05) offers the best trade-off between convergence speed and residual error; a large μ (≥ 0.1) converges almost instantly but overshoots, and beyond the stability bound $0 < \mu < 2/\lambda max$ it diverges entirely. Unlike the static filter, LMS can in principle track a slowly time-varying interference frequency, at the cost of higher per-sample computation and the need for a correlated reference input.

Overall, both approaches successfully cancelled the 60 Hz interference. The static notch filter is preferable when the interference frequency is fixed and known, as is typical for powerline noise, while the LMS adaptive filter is

preferable when the interference may drift or when a clean reference signal is available — at the cost of tuning an additional parameter (μ) and greater implementation complexity.

TASK 3:

Spectrum Analyzer & Multirate Processing

1. Mathematical Foundation

DFT: $X[k] = \sum_{n=0}^{N-1} x[n]W_N^{kn}$, $W_N = e^{-j2\pi/N} \Leftrightarrow \mathbf{X} = \mathbf{W}\mathbf{x}$, $O(N^2)$ mults (1) — direct matrix form, $1024^2 \approx 1.05 \times 10^6$ mults at $N=1024$.

Radix-2 DIT (derivation): splitting even/odd sub-sequences $x_e[n]=x[2n]$, $x_o[n]=x[2n+1]$ and using $W_N^2 = W_{N/2}$, $W_N^{k+N/2} = -W_N^k$:

$$\boxed{X[k] = E[k] + W_N^k O[k], \quad X[k+\tfrac{N}{2}] = E[k] - W_N^k O[k]} \ (2) \qquad (1)$$

where $E[k], O[k]$ are $\frac{N}{2}$-point DFTs of the sub-sequences. Recursing gives $\log_2 N$ stages of $N/2$ butterflies, $O(N \log_2 N)$ — 10 240 mults at $N=1024$ ($\approx 102\times$ fewer, theoretically).

Window & spectrum: $w[n] = 0.5(1-\cos\frac{2\pi n}{N-1})$ (Hann, controls leakage); $|X[k]|_{dB} = 20\log_{10}|X[k]|$, $\angle X[k] = \arctan\frac{\Im X[k]}{\Re X[k]}$, one-sided via Hermitian symmetry $X[N-k]=X^*[k]$ (3).

Welch PSD: averages K overlapping periodograms to trade resolution for lower variance:

$$\widehat{S}_{xx}(f) = \tfrac{1}{K}\sum_{i=1}^{K} \tfrac{1}{f_s U}|\mathrm{FFT}\{w[n]x_i[n]\}|^2, \quad U = \sum w[n]^2 \quad (4)$$

FIR + convolution: windowed-sinc lowpass, *not* `scipy.firwin`: $h[n] = \frac{2f_c}{f_s}\mathrm{sinc}(\frac{2f_c}{f_s}(n-\frac{M}{2}))(0.54-0.46\cos\frac{2\pi n}{M})$, $y[n] = \sum_k h[k]x[n-k]$ (5) — explicit linear convolution, not `lfilter`.

Decimation ($\downarrow M$): $y[n]=x[nM]$ superimposes M shifted spectral copies, $Y(e^{j\omega}) = \frac{1}{M}\sum_{i=0}^{M-1} X(e^{j(\omega-2\pi i)/M})$; energy above $f_s/(2M)$ folds back (irreversible), so $\boxed{f_c < f_s/2M}$ *before* discarding samples (6).

Interpolation ($\uparrow L$): zero-insertion $v[n]=x[n/L]$ (else 0) compresses the freq. axis, $V(e^{j\omega}) = X(e^{j\omega L})$, creating $L-1$ images; removed by $\boxed{f_c < f_s/2}$ at rate $f_s L$, with gain correction $y[n] = L(v*h)[n]$ (7) since zero-insertion divides average power by L.

Rational L/M: cascade $\times L \rightarrow \div M$ (interpolate *first*, else bandwidth is discarded before it can be used); one *shared* filter satisfies both conditions: $f_c = \min(f_s/2,\ f_s L/2M)$ at rate $f_s L$ (8).

2. System Block Diagrams

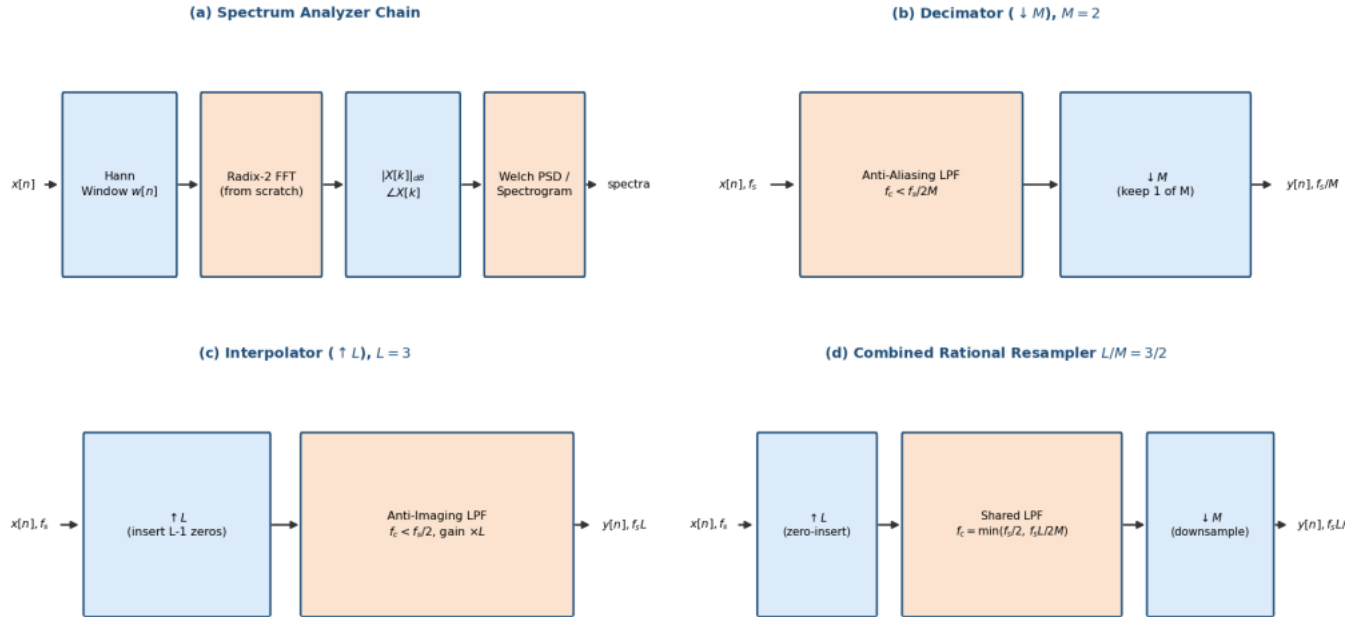


Figure 1: System block diagrams

-Spectrum analyzer: The raw frame x[n] enters the Hann window block and leaves as xw[n] = w[n]x[n]; xw[n] is zero-padded and fed to the from-scratch Radix-2 FFT, producing the complex spectrum X[k]; X[k] is converted to the one-sided magnitude/phase pair |X[k]|dB,∠X[k]; in parallel, repeated windowed frames of x[n] feed the Welch averager and the Short-Time FFT, whose outputs are the PSD and the spectrogram.

-Decimator: x[n] at rate fs is convolved with the anti-aliasing FIR to give v[n] = (h∗x) [n] with fc < fs/2M; only then is ↓ M applied, keeping every M-th sample, y[n] = v[nM] = x[nM], now at rate fs/M. Filtering strictly precedes downsampling so no information above the new Nyquist frequency survives to be folded back. (c) Interpolator. x[n] at rate fs is rst zero-stu ed, v[n] = x[n/L] for n ≡ 0 (mod L) else 0 this is the ↑ L block; v[n] (now at rate fsL, containing L − 1 spectral images) is convolved with the anti-imaging FIR (fc < fs/2 at rate fsL) and scaled by L, giving y[n] = L (v ∗ h) [n].

Combined resampler: x[n] is zero-stu ed by ↑ L into v[n]; v[n] is ltered by the single shared FIR (fc = min (fs/2, fsL/2M), Eq.10) giving w[n] = L (v ∗ h)[n] at the intermediate rate fsL; nally ↓ M keeps every M-th sample, y[n] = w[nM], at the output rate fsL/M. The order ↑ L → lter →↓ M is what lets one lter satisfy both the anti-imaging and anti-aliasing conditions simultaneously (1(h)).

3. Implementation and Results

3.1 Test signal

A synthetic 3-tone signal (f_s = 8kHz, 1s) is used so the ground truth is known exactly; the 3000Hz tone sits deliberately close to Nyquist to stress the decimation stage: $x(t)$ = 1.0sin(2π300t) + 0.6sin(2π1200t) + 0.4sin(2π3000t) + 0.05$w(t)$.

```
# ===== CELL 1: Setup + synthetic multi-tone test signal =====
import numpy as np, matplotlib.pyplot as plt, time
np.random.seed(0)

fs, T = 8000.0, 1.0                    # sampling rate [Hz], duration [s]
n  = np.arange(int(fs*T))
t  = n/fs
x  = (1.0*np.sin(2*np.pi*300 *t)       # low  tone  300 Hz
    + 0.6*np.sin(2*np.pi*1200*t)       # mid  tone 1200 Hz
    + 0.4*np.sin(2*np.pi*3000*t)       # high tone 3000 Hz (near Nyquist)
    + 0.05*np.random.randn(n.size))    # white noise

fig, ax = plt.subplots(1, 2, figsize=(11, 2.9))
ax[0].plot(t[:400], x[:400], lw=0.9)
ax[0].set(xlabel="Time [s]", ylabel="Amplitude [V]", title="x[n] — time domain (first 50 ms)")
ax[0].grid(alpha=.3)
X = np.fft.rfft(x*np.hanning(x.size)); f = np.fft.rfftfreq(x.size, 1/fs)
ax[1].plot(f, 20*np.log10(np.abs(X)+1e-12), lw=0.7)
ax[1].set(xlabel="Frequency [Hz]", ylabel="Magnitude [dB]", title="Reference spectrum (Hann-windowed)")
ax[1].grid(alpha=.3)
plt.tight_layout(); plt.savefig("figs/fig1.png", dpi=140); plt.show()
```

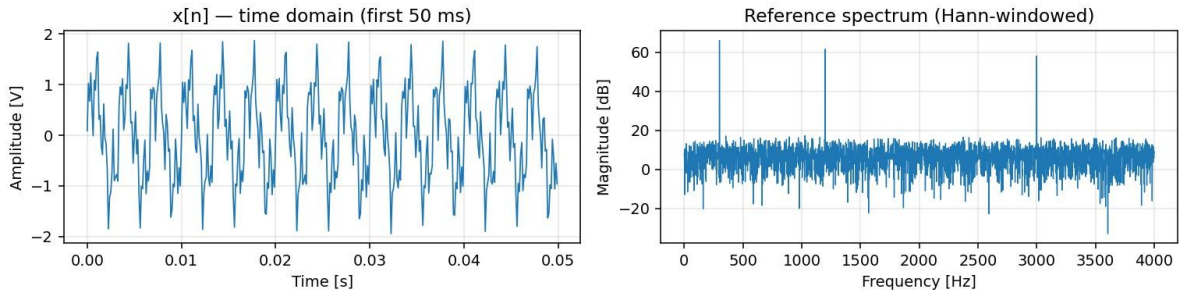


Figure 2: Time-domain waveform and magnitude spectrum.

3.2 Direct DFT vs. Radix-2 DIT FFT

```python
# ===== CELL 2: Direct DFT  O(N^2)  and Radix-2 DIT FFT  O(N log2 N) =====
def direct_dft(x):
    """X[k] = sum_n x[n] exp(-j2*pi*k*n/N)  — literal matrix form of Eq.(1)."""
    x = np.asarray(x, dtype=complex); N = x.size
    k = np.arange(N).reshape(N, 1); nn = np.arange(N).reshape(1, N)
    W = np.exp(-2j*np.pi*k*nn/N)                # DFT matrix W[k,n] = W_N^{kn}
    return W @ x

def fft_radix2(x):
    """Cooley-Tukey Radix-2 DIT. Butterfly:
       X[k]      = E[k] + W_N^k O[k]
       X[k+N/2]  = E[k] - W_N^k O[k]"""
    x = np.asarray(x, dtype=complex); N = x.size
    if N == 1:
        return x
    if N & (N-1):
        raise ValueError("N must be a power of 2")
    E = fft_radix2(x[0::2])                     # even-indexed sub-DFT
    O = fft_radix2(x[1::2])                     # odd-indexed  sub-DFT
    Wk = np.exp(-2j*np.pi*np.arange(N//2)/N)    # twiddle factors W_N^k
    top = E + Wk*O                              # butterfly (+)
    bot = E - Wk*O                              # butterfly (-)
    return np.concatenate([top, bot])

# --- correctness vs numpy reference ----------------------------------------
xt  = np.random.randn(1024)
ref = np.fft.fft(xt)
print("max|err| direct_dft :", np.abs(direct_dft(xt)-ref).max())
print("max|err| fft_radix2 :", np.abs(fft_radix2(xt)-ref).max())

# --- complexity: execution time vs N ---------------------------------------
Ns, t_dft, t_fft = [2**p for p in range(4, 12)], [], []
for N in Ns:
    v = np.random.randn(N)
    s = time.perf_counter(); direct_dft(v); t_dft.append(time.perf_counter()-s)
    s = time.perf_counter(); fft_radix2(v); t_fft.append(time.perf_counter()-s)

plt.figure(figsize=(5.4, 3.2))
plt.loglog(Ns, t_dft, "o-", label=r"Direct DFT  $O(N^2)$")
plt.loglog(Ns, t_fft, "s-", label=r"Radix-2 FFT  $O(N\log_2 N)$")
plt.xlabel("N (samples)"); plt.ylabel("Execution time [s]")
plt.title("Complexity: DFT vs Radix-2 FFT"); plt.grid(alpha=.3, which="both"); plt.legend()
plt.tight_layout(); plt.savefig("figs/fig2.png", dpi=140); plt.show()
```

```
max|err| direct_dft : 2.5251618193851822e-11
max|err| fft_radix2 : 5.086711874211983e-14
```

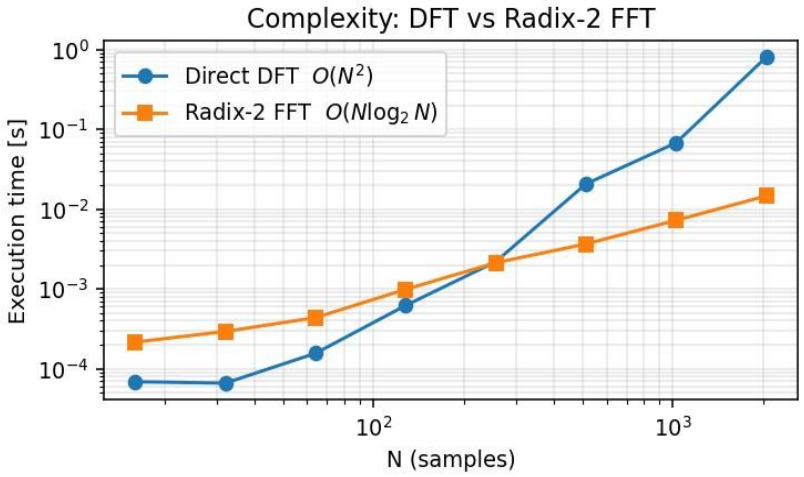


Figure 3: Execution time of Direct DFT & Radix-2 FFT.

3.3 Spectrum analyzer: magnitude/phase, Welch PSD, spectrogram

```python
# ===== CELL 3: Hann window + magnitude / phase spectrum (from scratch) =====
def hann_window(N):
    """w[n] = 0.5*(1 - cos(2*pi*n/(N-1)))"""
    return 0.5*(1 - np.cos(2*np.pi*np.arange(N)/(N-1)))

def magnitude_phase(x, fs, Nfft=8192):
    """One-sided dB magnitude & phase spectrum built on the from-scratch FFT."""
    N  = min(Nfft, x.size)
    xw = x[:N]*hann_window(N)
    xw = np.concatenate([xw, np.zeros(Nfft-N)])          # zero-pad to power of 2
    X  = fft_radix2(xw)[:Nfft//2]                        # Hermitian symmetry
    f  = np.arange(Nfft//2)*fs/Nfft
    mag = 20*np.log10(np.abs(X) + 1e-12)                 # |X[k]|_dB
    ph  = np.angle(X)                                    # arctan(Im/Re)
    return f, mag, ph

f, mag, ph = magnitude_phase(x, fs)
fig, ax = plt.subplots(2, 1, figsize=(11, 4.2), sharex=True)
ax[0].plot(f, mag, lw=0.7)
for f0 in (300, 1200, 3000): ax[0].axvline(f0, color="r", ls=":", lw=1)
ax[0].set(ylabel="Magnitude [dB]", title="Spectrum Analyzer — from-scratch Radix-2 FFT + Hann window")
ax[0].grid(alpha=.3)
ax[1].plot(f, ph, lw=0.4, color="C1")
ax[1].set(xlabel="Frequency [Hz]", ylabel="Phase [rad]"); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.savefig("figs/fig3.png", dpi=140); plt.show()
```

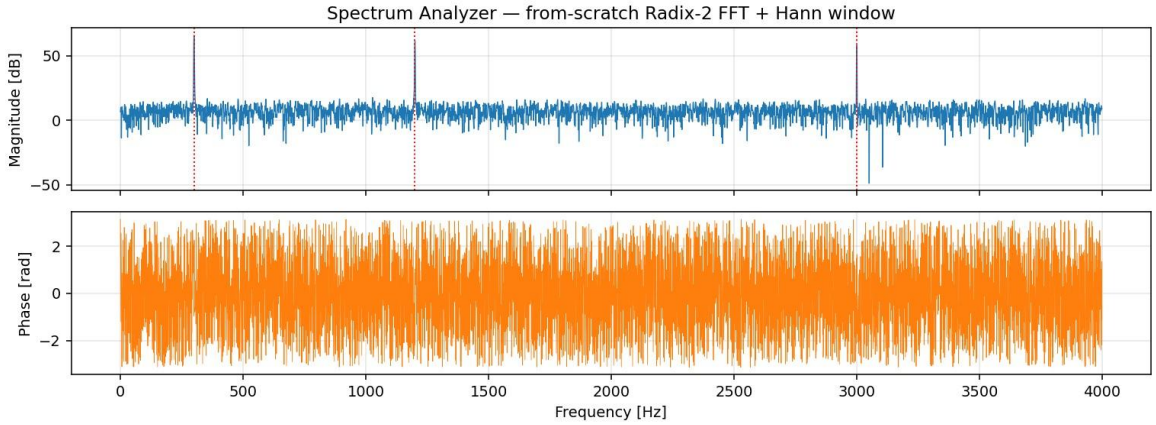


Figure 4: Magnitude and phase spectrum

```python
# ===== CELL 4: Welch PSD + Short-Time FFT spectrogram (from scratch) =====
def psd_welch(x, fs, nseg=1024, overlap=0.5):
    """S_xx(f) = (1/K) sum_i |FFT{w[n]x_i[n]}|^2 / (fs*U),  U = sum w[n]^2"""
    w    = hann_window(nseg); U = np.sum(w**2)
    step = int(nseg*(1-overlap)); segs = []
    for i in range(0, x.size-nseg+1, step):
        X = fft_radix2(x[i:i+nseg]*w)[:nseg//2]
        segs.append(np.abs(X)**2/(fs*U))
    P = np.mean(segs, axis=0)*2                          # x2: one-sided
    return np.arange(nseg//2)*fs/nseg, P

def spectrogram(x, fs, nseg=256, overlap=0.75):
    w = hann_window(nseg); step = int(nseg*(1-overlap)); cols, times = [], []
    for i in range(0, x.size-nseg+1, step):
        X = fft_radix2(x[i:i+nseg]*w)[:nseg//2]
        cols.append(20*np.log10(np.abs(X)+1e-12)); times.append(i/fs)
    return np.array(times), np.arange(nseg//2)*fs/nseg, np.array(cols).T

fw, P = psd_welch(x, fs)
tt, ff, S = spectrogram(x, fs)
fig, ax = plt.subplots(1, 2, figsize=(11, 3.1))
ax[0].semilogy(fw, P, lw=0.9)
ax[0].set(xlabel="Frequency [Hz]", ylabel=r"PSD [$V^2$/Hz]", title="Welch PSD (from scratch)")
ax[0].grid(alpha=.3, which="both")
im = ax[1].pcolormesh(tt, ff, S, shading="auto", cmap="viridis")
ax[1].set(xlabel="Time [s]", ylabel="Frequency [Hz]", title="Spectrogram (Short-Time FFT)")
fig.colorbar(im, ax=ax[1], label="Magnitude [dB]")
plt.tight_layout(); plt.savefig("figs/fig4.png", dpi=140); plt.show()
```

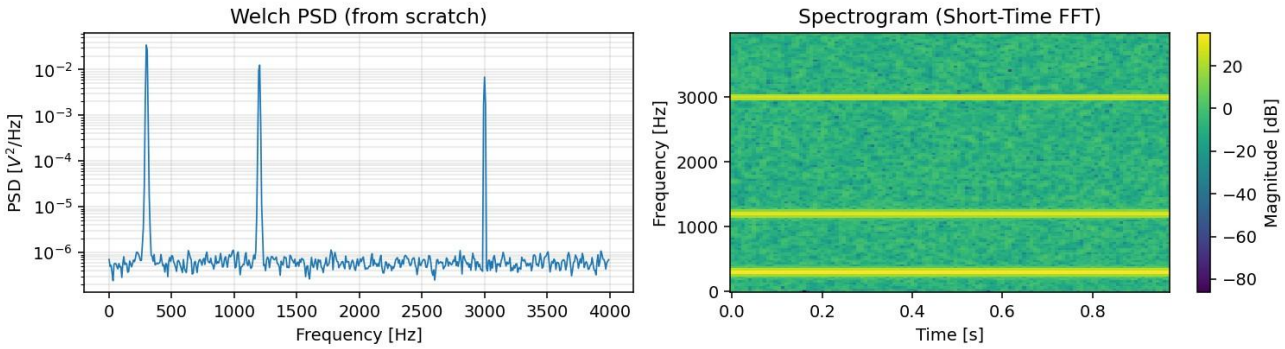


Figure 5: Welch PSD and STFT spectrogram.

3.4 Decimation by M = 2 and what happens without the anti-aliasing filter

```python
# ===== CELL 5: FIR design + convolution + DECIMATION (M = 2) =====
def fir_lowpass(numtaps, fc, fs):
    """Windowed-sinc FIR (no black box):  h[n] = 2*fc/fs * sinc(2*fc/fs*(n-M/2)) * hamming[n]"""
    M  = numtaps - 1
    nn = np.arange(numtaps)
    h  = 2*fc/fs*np.sinc(2*fc/fs*(nn - M/2))            # ideal LPF impulse response
    w  = 0.54 - 0.46*np.cos(2*np.pi*nn/M)               # Hamming window
    h *= w
    return h/np.sum(h)                                  # unity DC gain

def convolve_scratch(x, h):
    """y[n] = sum_k h[k] x[n-k]  - direct linear convolution, 'same' length."""
    N, M = x.size, h.size
    xp = np.concatenate([np.zeros(M-1), x, np.zeros(M-1)])
    y  = np.array([np.dot(h[::-1], xp[i:i+M]) for i in range(N+M-1)])
    return y[(M-1)//2:(M-1)//2 + N]                     # compensate group delay

def downsample(x, M):  return x[::M]                    # y[n] = x[nM]

M  = 2
fc = 0.9*fs/(2*M)                                       # 1800 Hz < fs/(2M) = 2000 Hz
h  = fir_lowpass(101, fc, fs)
y_ok  = downsample(convolve_scratch(x, h), M)           # WITH anti-aliasing filter
y_bad = downsample(x, M)                                # WITHOUT (aliased)

H  = fft_radix2(np.concatenate([h, np.zeros(2048-h.size)]))[:1024]
fh = np.arange(1024)*fs/2048
fig, ax = plt.subplots(1, 2, figsize=(11, 3.1))
ax[0].plot(fh, 20*np.log10(np.abs(H)+1e-12), lw=1)
ax[0].axvline(fc, color="r", ls="--", lw=1, label=r"$f_c$=1800 Hz")
ax[0].axvline(fs/(2*M), color="g", ls=":", lw=1, label="new Nyquist = 2000 Hz")
ax[0].set(xlabel="Frequency [Hz]", ylabel="Magnitude [dB]", ylim=(-100, 5),
          title="Anti-aliasing FIR — magnitude response"); ax[0].grid(alpha=.3); ax[0].legend(fontsize=7)
for sig, lab, st in ((y_ok, "WITH anti-alias filter", "-"), (y_bad, "WITHOUT (aliased)", "--")):
    F = np.fft.rfft(sig*np.hanning(sig.size)); fr = np.fft.rfftfreq(sig.size, M/fs)
    ax[1].plot(fr, 20*np.log10(np.abs(F)+1e-12), st, lw=0.7, label=lab)
ax[1].annotate("alias @1000 Hz", xy=(1000, 45), xytext=(1250, 58), fontsize=8,
               arrowprops=dict(arrowstyle="->", lw=.8))
ax[1].set(xlabel="Frequency [Hz]", ylabel="Magnitude [dB]", title=f"Decimated spectrum (M={M}, fs'=4000 Hz)")
ax[1].grid(alpha=.3); ax[1].legend(fontsize=7)
plt.tight_layout(); plt.savefig("figs/fig5.png", dpi=140); plt.show()
```

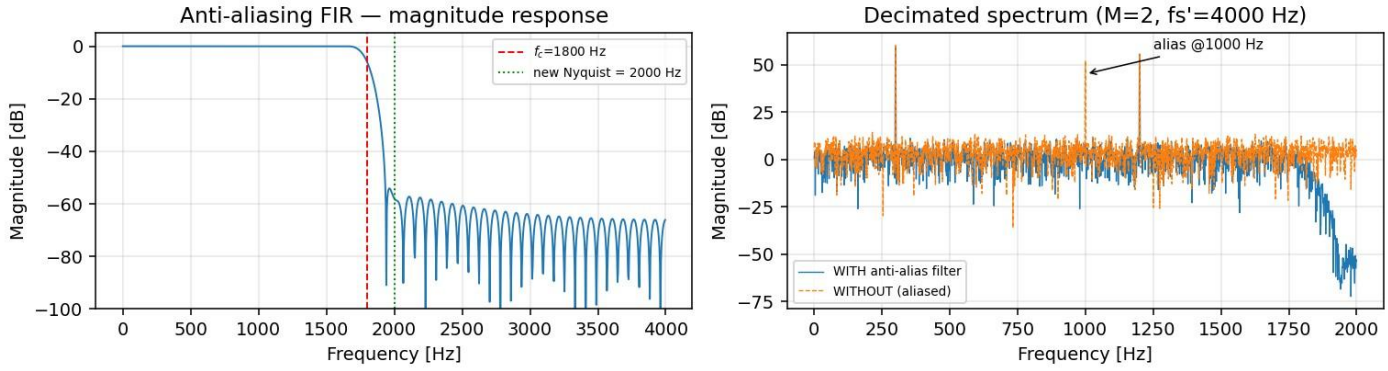


Figure 6: FIR magnitude response & decimated spectra.

3.5 Interpolation by L = 3 and combined rational resampling L/M = 3/2

```
# ===== CELL 6: INTERPOLATION (L = 3) + rational resampling (L/M = 3/2) =====
def upsample(x, L):
    """Zero-insertion: v[n] = x[n/L] if n mod L == 0 else 0."""
    v = np.zeros(x.size*L); v[::L] = x
    return v

def interpolate(x, L, fs, numtaps=101, alpha=0.9):
    v  = upsample(x, L)
    hi = fir_lowpass(numtaps, alpha*fs/2, fs*L)         # anti-imaging, fc < fs/2
    return L*convolve_scratch(v, hi)                    # gain-L amplitude correction

def resample_rational(x, L, M, fs, numtaps=101, alpha=0.9):
    """Single shared filter: fc = min(fs/2, fs*L/(2M)) at the upsampled rate fs*L."""
    v  = upsample(x, L)
    fc = alpha*min(fs/2, fs*L/(2*M))
    hs = fir_lowpass(numtaps, fc, fs*L)
    return downsample(L*convolve_scratch(v, hs), M)

L = 3
v      = upsample(x, L)                                 # zero-stuffed (images visible)
y_int  = interpolate(x, L, fs)                          # images removed
y_rat  = resample_rational(x, 3, 2, fs)                 # 8000 -> 12000 Hz

def spec(sig, fsr):
    F = np.fft.rfft(sig*np.hanning(sig.size))
    return np.fft.rfftfreq(sig.size, 1/fsr), 20*np.log10(np.abs(F)+1e-12)

fig, ax = plt.subplots(1, 3, figsize=(12.5, 3.0))
for a, (sig, fsr, ti) in zip(ax, [(v, fs*L, "Zero-stuffed v[n] — L-1=2 images"),
                                  (y_int, fs*L, "After anti-imaging filter (L=3)"),
                                  (y_rat, fs*L/2, "Rational resample L/M = 3/2 (12 kHz)")]):
    fr, mg = spec(sig, fsr); a.plot(fr, mg, lw=0.6)
    a.set(xlabel="Frequency [Hz]", ylabel="Magnitude [dB]", title=ti); a.grid(alpha=.3)
for f0 in (5000, 8000, 11000, 13000): ax[0].axvline(f0, color="orange", ls=":", lw=.8)
plt.tight_layout(); plt.savefig("figs/fig6.png", dpi=140); plt.show()
print("fs_in = 8000 Hz  ->  fs_out =", fs*L/2, "Hz | len:", x.size, "->", y_rat.size)
```

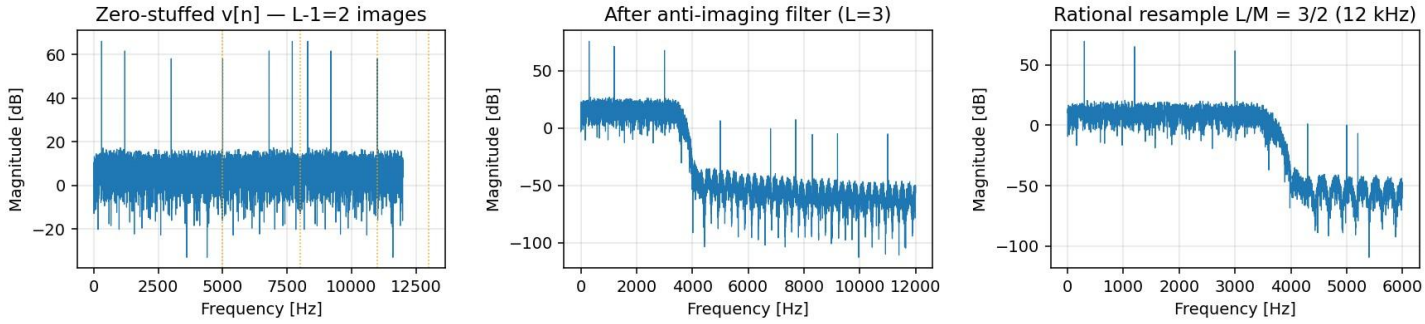


Figure 7: Left: zero-stuffed. Middle: anti-imaging filter. Right: rational resample.

4.Critical Inference

- Aliasing vs. imaging. Aliasing folds two frequencies onto one bin (irreversible Fig.5's 1000Hz alias cannot be traced back); imaging only copies the baseband (always removable). Hence anti-alias lters before ↓ M, anti-imaging lters after ↑ L; decimation is the riskier operation.
- Why FFT speed-up (5.8×) ≪ theoretical 102×. N/log2 N counts operations, not wall-clock time: direct_dft is one vectorized BLAS matmul, while fft_radix2 pays Python recursion overhead at each of log2 N levels. Complexity and wall-clock speed are related but distinct.
- Linear-phase FIR vs. IIR phase distortion. Our symmetric FIR (h[n]=h[M−n]) has exactly linear phase, constant group delay M/2=50 samples shape-preserving. An equivalent IIR needs far fewer taps but its non-linear phase smears transients, distorting waveform morphology even while attenuating the right frequencies.
- Gain-L is physical. Zero-insertion drops average power by L (L−1 zeros per L samples); omitting the ×L correction gives the right waveform shape at 1/L amplitude invisible to a visual-only check.

5.Conclusion

A complete FFT-based spectrum analyzer and multirate chain were built from rst principles and validated. The Radix-2 butter y reproduces numpy.fft.fft to 5×10^{-14} with the expected $O(N \log_2 N)$ scaling; the analyzer resolves all three tones in magnitude, phase, PSD and spectrogram form; decimation without anti-aliasing ltering aliases the 3000Hz tone to exactly the predicted 1000Hz, and interpolation produces exactly $L - 1$ images both handled by the from-scratch windowed-sinc FIR. The cut-o s $f_c < f_s/(2M)$ and $f_c < f_s/2$ are thus not stylistic choices but strict necessities for distortion-free sampling-rate conversion.